

Guia dos Labs — Desenvolvendo Software com IA Generativa

Compilado dos 7 labs da disciplina (LAB-000 a LAB-006) com objetivos, fluxo, comandos e troubleshooting. Use como referência consultável durante os blocos de laboratório nas tardes dos Dias 1 e 2.

Convenção: cada lab é uma seção independente. Os notebooks executáveis correspondentes ficam em `notebooks/NN-*.ipynb`. Se algo divergir entre o guia e o notebook, o **notebook é a fonte da verdade**.

Sumário

- LAB-000 — [Lab 0: Cliente LLM Robusto](#)
- LAB-001 — [Lab 1: Agent CLI Single-Turn com Tool-use — Calculadora + Consulta de Documentação](#)
- LAB-002 — [Lab 2: Pipeline RAG End-to-End com Chroma](#)
- LAB-003 — [Lab 3: Suite de Avaliação Automatizada com RAGAS](#)
- LAB-004 — [Lab 4: Diagnóstico de Falhas RAG](#)
- LAB-005 — [Lab 5: Semantic Cache + Cheap-First Model Routing](#)
- LAB-006 — [Lab 6: Projeto Portfólio Integrador](#)

Lab 0: Cliente LLM Robusto

Tipo: Guiado | **Duração:** 45 min | **Nível:** Iniciante (warm-up de M1) | **Bloom:** Apply

Objetivos de Aprendizagem:

- **M1-O3:** Implementar integração programática com APIs de LLM (OpenAI/Anthropic) usando streaming, retries e saída estruturada validada com Pydantic

Posição no curso: rodar **antes** do LAB-001. Este lab é o piso técnico — sem ele, alunos da trilha **Basic** chegam ao tool-use sem dominar as primitivas (Pydantic, streaming, retry). Trilhas Intermediate/Advanced podem fazer skim em ~20 min.

Fluxo do Lab



Fluxo do Lab 0: setup, warm-up, Pydantic structured output, streaming, retry com backoff e error handling

Pré-requisitos

Requisito	Versão
Python	3.10+
openai (SDK)	latest
pydantic	2.x
python-dotenv	1.x

Etapa 1 — Warm-up: 1 call simples

Antes de adicionar complexidade, garanta que o cliente conecta:

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ['GEMINI_API_KEY'],
    base_url='https://generativelanguage.googleapis.com/v1beta/openai/',
)
MODEL = 'gemini-2.5-flash-lite'

resp = client.chat.completions.create(
    model=MODEL,
    messages=[{'role': 'user', 'content': 'Diga apenas: ping'}],
    temperature=0.0,
)
print(resp.choices[0].message.content)
```

Ponto de verificação: deve imprimir `ping` (ou variação curta). Se erro 401/403 → key inválida; se 429 → rate limit (esperar 60s).

Etapa 2 — Pydantic Structured Output

Em vez de parsear texto livre do LLM, **force JSON validado**.

```
from pydantic import BaseModel, Field
from typing import Literal
import json

class Classification(BaseModel):
    sentiment: Literal['positivo', 'negativo', 'neutro']
    confidence: float = Field(ge=0, le=1)
    reasoning: str = Field(max_length=200)

def classify(text: str) -> Classification:
    schema = Classification.model_json_schema()
    prompt = f"""Classifique o sentimento da mensagem abaixo.
    Responda APENAS em JSON valido com este schema:
    {json.dumps(schema, indent=2)}

    Mensagem: {text}
    Output: """

    resp = client.chat.completions.create(
        model=MODEL,
        messages=[{'role': 'user', 'content': prompt}],
        response_format={'type': 'json_object'},
        temperature=0.0,
    )
    raw = resp.choices[0].message.content
    return Classification.model_validate_json(raw)

result = classify('Adorei o atendimento, super atencioso!')
print(f'sentiment={result.sentiment}, confidence={result.confidence}')
print(f'reasoning: {result.reasoning}')
```

Reflexão: o que acontece se o LLM responder com JSON que NÃO segue o schema (e.g., `confidence=1.5`)?

Resposta: `model_validate_json()` levanta `ValidationError`. Isso é **bom** — você quer detectar drift do modelo cedo, não passar dados inválidos para produção.

Etapa 3 — Streaming

Streaming reduz **time-to-first-token** drasticamente — UX percebe ~3× mais rápido mesmo com mesmo throughput total.

```
def stream_completion(prompt: str) -> str:
    stream = client.chat.completions.create(
        model=MODEL,
        messages=[{'role': 'user', 'content': prompt}],
        stream=True,
    )

    full_text = ''
    for chunk in stream:
        delta = chunk.choices[0].delta.content or ''
        print(delta, end='', flush=True)  # renderiza ao vivo
        full_text += delta
    print()  # newline final
    return full_text

_ = stream_completion('Liste 5 frameworks Python para LLM, 1 linha cada')
```

Ponto de verificação: você deve ver o texto aparecer **palavra por palavra**, não bloco inteiro.

Reflexão: streaming **não combina bem** com retry. Por quê?

Resposta: se a conexão cai no meio do stream, você já mostrou parte ao usuário. Retry vai gerar conteúdo **diferente** (LLM não-determinístico), causando “flicker” ou continuação inconsistente. Estratégias: - Para chat UI: aceitar e exibir “[reconectando]” - Para batch processing: não usar streaming, usar retry idempotente - Streaming + retry idempotente: só funciona se o servidor expor cursor (raro)

Etapa 4 — Retry com Exponential Backoff + Jitter

Production-ready: chamadas falham por rate limit, network, server-5xx. Não retry → produto frágil. Retry sem backoff → thundering herd.

```

import random
import time
from openai import (
    APIConnectionError,
    APITimeoutError,
    RateLimitError,
    InternalServerError,
    AuthenticationError,
    BadRequestError,
)

RETRYABLE = (RateLimitError, APIConnectionError, APITimeoutError, InternalServerError)
NON_RETRYABLE = (AuthenticationError, BadRequestError)

def call_with_retry(prompt: str, max_attempts: int = 3) -> str:
    base_delay = 1.0
    last_error: Exception | None = None

    for attempt in range(1, max_attempts + 1):
        try:
            resp = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': prompt}],
                temperature=0.0,
            )
            return resp.choices[0].message.content or ''

        except RETRYABLE as e:
            last_error = e
            if attempt == max_attempts:
                break

            delay = base_delay * (2 ** (attempt - 1))
            jitter = random.uniform(0, delay * 0.5)
            wait = delay + jitter
            print(f' [retry {attempt}] {type(e).__name__}: aguardando {wait:.1f}s')
            time.sleep(wait)

        except NON_RETRYABLE as e:
            # Auth invalida ou input malformado — re-tentar não adianta
            raise

    raise RuntimeError(f'Max retries ({max_attempts}) excedidas') from last_error

## Teste manual (so cai em erro real se voce derrubar a internet):
result = call_with_retry('Diga: ok')
print(f'Resultado: {result}')

```

3 elementos críticos do retry:

1. **Distinção retryable vs não-retryable** — auth nunca vira, bad request nunca vira
2. **Exponential backoff** — delays 1s, 2s, 4s, 8s (não linear)
3. **Jitter** — soma random ao delay para evitar 10 clientes batendo no mesmo instante

Etapa 5 — Combinar: Pipeline Robusto

Juntar Pydantic + Retry em uma função única:

```

def robust_classify(text: str, max_attempts: int = 3) -> Classification:
    schema = Classification.model_json_schema()
    prompt = f"""Classifique o sentimento. Responda apenas em JSON:
Schema: {json.dumps(schema)}
Mensagem: {text}
Output: """

    base_delay = 1.0
    for attempt in range(1, max_attempts + 1):
        try:
            resp = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': prompt}],
                response_format={'type': 'json_object'},
                temperature=0.0,
            )
            raw = resp.choices[0].message.content or ''
            return Classification.model_validate_json(raw)

        except RETRYABLE as e:
            if attempt == max_attempts:
                raise

            wait = base_delay * (2 ** (attempt - 1)) + random.uniform(0, 0.5)
            print(f' [retry {attempt}] {type(e).__name__}: {wait:.1f}s')
            time.sleep(wait)

        except NON_RETRYABLE:
            raise

    raise RuntimeError('unreachable')

## Bateria de 3 mensagens
for msg in [
    'Adorei o atendimento, super atencioso!',
    'Demoraram 3 dias pra responder e nada.',
    'Recebi o produto.',
]:
    out = robust_classify(msg)
    print(f'{msg[:40]:42s} -> {out.sentiment} ({out.confidence:.2f})')

```

Verificação

Warm-up: primeira call funciona, key válida

Pydantic: `Classification.model_validate_json()` aceita JSON válido e rejeita inválido

Streaming: texto aparece palavra-por-palavra, não bloco

Retry: entende a diferença entre retryable (rate limit, network) e non-retryable (auth, bad request)

Pipeline combinado: 3 mensagens classificadas com sucesso

Troubleshooting

PROBLEMA 1: `VALIDATIONERROR` EM QUASE TODA CHAMADA

Causa: LLM não está respeitando o schema.

Solução: 1. Reforce no prompt: "Responda APENAS em JSON. Sem prefixos, sem comentarios." 2. Use `response_format={'type': 'json_object'}` se o provider suporta 3. Reduza `temperature` para 0.0 4. Adicione 1-2 few-shot examples mostrando o output exato

PROBLEMA 2: STREAMING "TRAVA" NO MEIO

Causa: conexão idle timeout, frequente em redes corporativas.

Solução: adicionar `timeout=60` ao client OU detectar via `time.time()` que >30s sem chunk e reconectar (não retry idempotente — só

log e segue).

PROBLEMA 3: RETRY LOOP INFINITO MESMO COM `MAX_ATTEMPTS`

Causa: o erro NÃO está em `RETRYABLE` mas você está capturando-o em outro `except` que continua.

Solução: o `raise` final em `NON_RETRYABLE` é essencial — sem ele, o erro é silenciado.

Próximo Passo

Notebook 00 fechado → **abrir notebook 01** (LAB-001 Agent CLI). As primitivas deste lab (Pydantic, retry, structured output) reaparecem lá compondo o agente com tool-use.

Referências

- [OpenAI Python SDK — Errors and Retries](#)
- [Pydantic v2 — Models](#)
- [Exponential Backoff and Jitter — AWS Architecture Blog](#)

Gerado por `/edu-lab-designer` — 2026-05-18 (P1 da iteração de coverage-report)

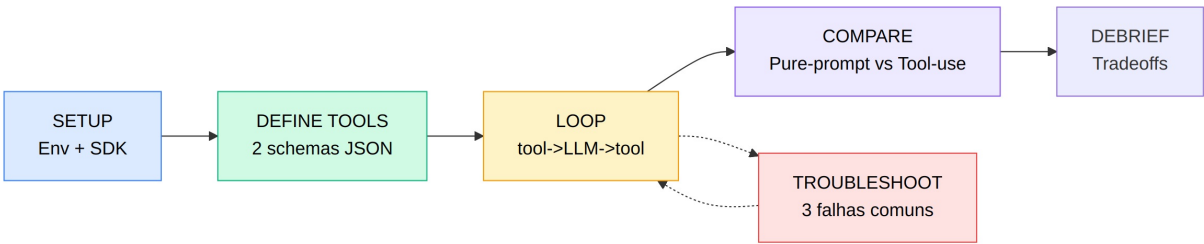
Lab 1: Agent CLI Single-Turn com Tool-use — Calculadora + Consulta de Documentação

Tipo: Projeto | **Duração:** 90 min | **Nível:** Intermediário | **Bloom:** Apply

Objetivos de Aprendizagem:

- **M1-04:** Implementar function-calling (tool-use) com schema JSON e loop de execução de ferramentas, criando um agente single-turn
- **M1-05:** Comparar abordagens de pure-prompt vs. tool-use para um mesmo problema, identificando quando cada uma é apropriada

Fluxo do Lab



Fluxo do Lab 1: setup, definição de 2 tools, loop tool-use single-turn e comparativo pure-prompt vs tool-use

Pré-requisitos

Requisito	Versão
Python	3.10+
openai (SDK)	latest
pydantic	2.x
python-dotenv	1.x

Default da turma: **Gemini Free Tier**. Alternativas (OpenAI, Anthropic, Ollama local) em `notebooks/API-KEYS.pdf`.

Setup (escolha 1 provider)

Inicializar projeto:

```
uv venv && source .venv/bin/activate
uv pip install openai pydantic python-dotenv
```

Usamos o SDK `openai` para os 3 providers (todos expõem endpoint OpenAI-compatible). Apenas o `base_url` e o modelo mudam.

Opção A — Gemini Free Tier (default):

```
echo 'GEMINI_API_KEY=Aiza...' >> .env
```

```
import json
import os
from typing import Any
from openai import OpenAI
from pydantic import BaseModel, Field

client = OpenAI(
    api_key=os.environ["GEMINI_API_KEY"],
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)

MODEL = "gemini-2.5-flash-lite" # 15 RPM, 1000 RPD; troque para gemini-2.5-flash se precisar de melhor too
```

Opção B — OpenAI (\$5 free credit):

```
client = OpenAI() # OPENAI_API_KEY do .env
MODEL = "gpt-4o-mini"
```

Opção C — Ollama 100% local (Qwen3 é o mais estável em tool-use):

```
ollama pull qwen3:8b && ollama serve
```

```
client = OpenAI(
    api_key="ollama", # qualquer string
    base_url="http://localhost:11434/v1",
)

MODEL = "qwen3:8b"
```

Atenção tool-use: Gemini Flash-Lite ocasionalmente decide não chamar a tool quando a query é ambígua. Se isso acontecer no lab, suba para `gemini-2.5-flash` ou force `tool_choice` (ver Troubleshooting §1).

Etapa 1 — Definir 2 Tools com Schema JSON

Construir schemas para `calculator` (eval seguro) e `lookup_doc` (busca em corpus local).

```

def calculator(expression: str) -> str:
    """Avalia expressao aritmetica simples."""
    allowed = set("0123456789+-*/(). ")
    if not all(c in allowed for c in expression):
        return "ERROR: expressao contem caracteres nao permitidos"
    try:
        return str(eval(expression, {"__builtins__": {}}, {}))
    except Exception as e:
        return f"ERROR: {e}"

DOCS = {
    "retry": "Use exponential backoff entre tentativas para evitar thundering herd.",
    "pydantic": "Pydantic valida payload via BaseModel + type hints.",
    "streaming": "Streaming reduz time-to-first-token mas dificulta retry idempotente.",
}

def lookup_doc(term: str) -> str:
    """Consulta documentacao local por termo exato (case-insensitive)."""
    return DOCS.get(term.lower(), f"NOT_FOUND: termo '{term}' nao encontrado")

TOOLS = [
    {
        "type": "function",
        "function": {
            "name": "calculator",
            "description": "Avalia uma expressao aritmetica simples (apenas + - * / e parenteses).",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "Expressao matematica, ex: '12 * (3 + 4)'",
                    }
                },
                "required": ["expression"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "lookup_doc",
            "description": "Consulta um termo na base de documentacao local (retry, pydantic, streaming).",
            "parameters": {
                "type": "object",
                "properties": {
                    "term": {
                        "type": "string",
                        "description": "Termo a buscar, ex: 'retry'",
                    }
                },
                "required": ["term"],
            },
        },
    },
]

TOOL_REGISTRY = {"calculator": calculator, "lookup_doc": lookup_doc}

```

Reflexão: Por que o `calculator` valida o input antes de `eval` ? Que ataque essa validação previne?

Etapa 2 — Loop tool→LLM→tool (Single-Turn Agent)

Implementar o loop: LLM decide tool → executa → retorna resultado → LLM finaliza.

```
def run_agent(user_query: str, max_iters: int = 5) -> str:
    messages = [
        {
            "role": "system",
            "content": (
                "Voce e um assistente que pode usar tools. "
                "Use 'calculator' para contas e 'lookup_doc' para definicoes tecnicas. "
                "Sempre cite a fonte quando usar lookup_doc."
            ),
        },
        {"role": "user", "content": user_query},
    ]

    for _ in range(max_iters):
        response = client.chat.completions.create(
            model=MODEL,
            messages=messages,
            tools=TOOLS,
            tool_choice="auto",
        )
        msg = response.choices[0].message
        messages.append(msg.model_dump(exclude_unset=True))

        if not msg.tool_calls:
            return msg.content or ""

        for call in msg.tool_calls:
            fn_name = call.function.name
            args = json.loads(call.function.arguments)
            result = TOOL_REGISTRY[fn_name](**args)
            messages.append(
                {
                    "role": "tool",
                    "tool_call_id": call.id,
                    "content": result,
                }
            )

    return "[ERROR] max iterations excedidas"

print(run_agent("Quanto e 47 * 13 + 200?"))
print(run_agent("O que e retry em chamadas HTTP?"))
print(run_agent("Calcule 25% de 480 e me explique o que e pydantic"))
```

Ponto de verificação: As 3 queries devem produzir respostas corretas ($47 \cdot 13 + 200 = 811$, definição de retry, 120 + definição de pydantic). A 3a query exige **2 tool calls** consecutivas.

Etapa 3 — Comparativo Pure-Prompt vs Tool-use

Repetir as mesmas 3 queries **sem tools**, apenas com prompt:

```
def run_pure_prompt(user_query: str) -> str:
    response = client.chat.completions.create(
        model=MODEL,
        messages=[
            {"role": "system", "content": "Responda diretamente. Faça contas mentalmente."},
            {"role": "user", "content": user_query},
        ],
    )
    return response.choices[0].message.content or ""

print(run_pure_prompt("Quanto e 47 * 13 + 200?"))
print(run_pure_prompt("O que e retry em chamadas HTTP?"))
print(run_pure_prompt("Calcule 25% de 480 e me explique o que e pydantic"))
```

Reflexão: Compare as 6 respostas (3 com tool, 3 sem). Em qual delas o LLM puro errou ou alucinou? Em qual o tool-use foi overkill?

Verificação

Checklist de conclusão do lab:

Schemas JSON definidos para `calculator` e `lookup_doc` com `parameters.required` populado

TOOL_REGISTRY mapeia nomes para funções Python executáveis

Loop tool→LLM→tool termina ou por resposta final ou por `max_iters`

3 queries de teste produzem resultados corretos via tool-use

Comparativo pure-prompt vs tool-use documentado em texto livre (mínimo 3 frases)

TABELA COMPARATIVA ESPERADA

Query	Pure-prompt	Tool-use	Recomendado
Aritmética com >2 dígitos	erro silencioso ocasional	sempre exato	Tool-use
Definição factual	depende do training cutoff	source-anchored	Tool-use
Conversa aberta	natural	overhead desnecessário	Pure-prompt

Troubleshooting

PROBLEMA 1: `TOOL_CALLS` É `NONE` **MESMO COM** `TOOL_CHOICE="AUTO"`

Sintoma: O LLM responde direto sem usar a tool, mesmo para conta complexa.

Causa: System prompt fraco; LLM não percebe que tem permissão para chamar tool.

Solução: Reforçar no system: "Sempre use 'calculator' para QUALQUER calculo aritmetico." Ou usar `tool_choice={"type": "function", "function": {"name": "calculator"}}` para forçar.

PROBLEMA 2: `JSON.JSONDECODEERROR` **EM** `CALL.FUNCTION.ARGUMENTS`

Sintoma: Erro ao parsear argumentos da tool call.

Causa: LLM gerou JSON inválido (raro com modelos novos, comum com modelos antigos).

Solução: Envolver o `json.loads` em `try/except` e retornar erro como tool result (`"ERROR: invalid arguments"`) — o LLM faz auto-correção no próximo turno.

PROBLEMA 3: LOOP INFINITO (LLM CHAMA MESMA TOOL REPETIDAMENTE)

Sintoma: `max_iters` excedido sem resposta final.

Causa: Tool retornou erro mas LLM não interpretou e re-chamou.

Solução: Padronizar tool results começando com `OK:` ou `ERROR:` e instruir no system: "Se uma tool retornar ERROR, NAO repita a mesma chamada; resuma o erro ao usuario."

Referências

- [OpenAI Function Calling](#)
- [Anthropic Tool Use](#)
- [Pydantic v2 Docs](#)

Gerado por `/edu-lab-designer` — 2026-05-18

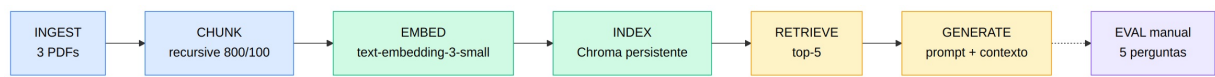
Lab 2: Pipeline RAG End-to-End com Chroma

Tipo: Guiado | Duração: 90 min | Nível: Intermediário | Bloom: Apply

Objetivos de Aprendizagem:

- **M2-O1:** Descrever as etapas de um pipeline RAG (chunk → embed → index → retrieve → augment → generate) e o papel de cada componente
- **M2-O2:** Implementar um pipeline RAG end-to-end em Python usando vector store (Chroma/FAISS) sobre um corpus de documentos reais

Fluxo do Lab



Fluxo do Lab 2: ingestão de 3 PDFs, chunking recursivo, embedding/indexing em Chroma, retrieval e generation

Pré-requisitos

Requisito	Versão
Python	3.10+
chromadb	0.5+
openai (SDK)	latest
pypdf	4.x
langchain-text-splitters	0.2+

Default: **Gemini Free** para LLM e embeddings; alternativa para embeddings 100% local com `sentence-transformers`. Setup completo em `notebooks/API-KEYS.pdf`.

Setup (escolha 1 stack)

```
uv pip install chromadb openai pypdf langchain-text-splitters python-dotenv
mkdir -p data/corpus
```

Baixar o corpus de 3 papers semanais (ver `datasets/README.pdf` da disciplina):

```
python datasets/download.py # baixa para data/corpus/ e valida SHA256
```

Corpus: “Attention Is All You Need” (Vaswani 2017), “Retrieval-Augmented Generation” (Lewis 2020), “Lost in the Middle” (Liu 2023). ~53 páginas / ~5.5 MB total → ~150 chunks após split.

```
import os
from pathlib import Path
from pypdf import PdfReader
from langchain_text_splitters import RecursiveCharacterTextSplitter
import chromadb
from openai import OpenAI
```

Opção A — Gemini Free Tier (default):

```
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

client = OpenAI(
    api_key=os.environ["GEMINI_API_KEY"],
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)
LLM_MODEL = "gemini-2.5-flash-lite"
EMBED_MODEL = "gemini-embedding-001"
embed_fn = OpenAIEmbeddingFunction(
    api_key=os.environ["GEMINI_API_KEY"],
    model_name=EMBED_MODEL,
    api_base="https://generativelanguage.googleapis.com/v1beta/openai/",
)
```

Opção B — OpenAI (\$5 free credit):

```
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

client = OpenAI()
LLM_MODEL = "gpt-4o-mini"
EMBED_MODEL = "text-embedding-3-small"
embed_fn = OpenAIEmbeddingFunction(
    api_key=os.environ["OPENAI_API_KEY"],
    model_name=EMBED_MODEL,
)
```

Opção C — Embeddings 100% local com `sentence-transformers` + LLM via Gemini ou Ollama:

```
uv pip install sentence-transformers
```

```
from chromadb.utils.embedding_functions import SentenceTransformerEmbeddingFunction

embed_fn = SentenceTransformerEmbeddingFunction(model_name="all-MiniLM-L6-v2")
## LLM: usar Opção A (Gemini) ou C (Ollama com qwen3:8b) – não precisa ser do mesmo provider
```

```
CORPUS_DIR = Path("data/corpus")
PERSIST_DIR = "data/chroma"
```

Etapa 1 — Ingestão de PDFs

Ler texto de cada PDF e preservar metadata (filename, página).

```
def ingest_pdfs(corpus_dir: Path) -> list[dict]:
    docs = []
    for pdf_path in sorted(corpus_dir.glob("*.pdf")):
        reader = PdfReader(pdf_path)
        for page_idx, page in enumerate(reader.pages):
            text = page.extract_text() or ""
            if text.strip():
                docs.append({
                    "text": text,
                    "source": pdf_path.name,
                    "page": page_idx + 1,
                })
    return docs

docs = ingest_pdfs(CORPUS_DIR)
print(f"Total de páginas ingeridas: {len(docs)}")
```

Ponto de verificação: Deve haver ≥ 10 entradas em `docs`. Inspecionar `docs[0]` para confirmar texto + metadata.

Etapa 2 — Chunking Recursivo

Quebrar páginas em chunks de 800 tokens com overlap de 100 (estratégia padrão para papers densos).

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=100,
    separators=["\n\n", "\n", ". ", " ", ""],
)

chunks = []
for doc in docs:
    splits = splitter.split_text(doc["text"])
    for i, chunk in enumerate(splits):
        chunks.append({
            "id": f"{doc['source']}-p{doc['page']}-c{i}",
            "text": chunk,
            "source": doc["source"],
            "page": doc["page"],
            "chunk_idx": i,
        })

print(f"Total de chunks: {len(chunks)}")
print(f"Tamanho medio: {sum(len(c['text']) for c in chunks)/len(chunks):.0f} chars")
```

Reflexão: Por que `chunk_overlap=100`? O que aconteceria com `overlap=0` em um chunk que termina no meio de uma frase importante?

Etapa 3 — Embedding e Indexing em Chroma

```

chroma_client = chromadb.PersistentClient(path=PERSIST_DIR)

# Reusa o embed_fn definido no Setup (Opcao A/B/C). NAO redefinir aqui:
# fixar OpenAIEmbeddingFunction quebraria o default Gemini e a opcao local.
collection = chroma_client.get_or_create_collection(
    name="papers",
    embedding_function=embed_fn,
)

collection.add(
    ids=[c["id"] for c in chunks],
    documents=[c["text"] for c in chunks],
    metadatas=[
        {"source": c["source"], "page": c["page"], "chunk_idx": c["chunk_idx"]}
        for c in chunks
    ],
)

print(f"Indexed: {collection.count()} chunks")

```

Ponto de verificação: `collection.count()` deve casar com `len(chunks)`. A pasta `data/chroma/` deve conter ~5-50MB.

Etapa 4 — Retrieval (top-5)

```

def retrieve(query: str, k: int = 5) -> list[dict]:
    result = collection.query(query_texts=[query], n_results=k)
    return [
        {
            "text": result["documents"][0][i],
            "source": result["metadatas"][0][i]["source"],
            "page": result["metadatas"][0][i]["page"],
            "distance": result["distances"][0][i],
        }
        for i in range(len(result["documents"][0]))
    ]

hits = retrieve("Qual e a diferenca entre fine-tuning e RAG?", k=5)
for h in hits:
    print(f"[{h['source']}:p{h['page']}] dist={h['distance']:.3f}")
    print(f"  {h['text'][:120]}...\n")

```

Reflexão: Os 5 chunks recuperados parecem relevantes para a pergunta? Como você confirmaria isso quantitativamente (preview de M2-O3)?

Etapa 5 — Augment + Generate

Construir prompt com contexto e gerar resposta.

```

RAG_PROMPT = """Voce e um assistente tecnico. Responda APENAS com base no contexto abaixo.
Se a informacao nao estiver no contexto, diga "Nao encontrado no corpus".
Sempre cite a fonte usando o formato [arquivo:pagina].

CONTEXTO:
{context}

PERGUNTA: {question}

RESPOSTA: """

def rag_answer(question: str, k: int = 5) -> dict:
    hits = retrieve(question, k=k)
    context = "\n\n--\n\n".join(
        f"[{h['source']}:p[{h['page']}]}\n{h['text']}" for h in hits
    )
    response = client.chat.completions.create(
        model=LLM_MODEL,
        messages=[{"role": "user", "content": RAG_PROMPT.format(context=context, question=question)}],
        temperature=0.0,
    )
    return {
        "answer": response.choices[0].message.content,
        "sources": [(h["source"], h["page"]) for h in hits],
    }

## Bateria de 5 perguntas
QUERIES = [
    "Qual e a diferenca entre fine-tuning e RAG?",
    "O que e chunking e por que importa?",
    "Que metricas avaliam qualidade de retrieval?",
    "Como reduzir alucinacao em LLM?",
    "Qual o papel de embeddings em busca semantica?",
]

for q in QUERIES:
    out = rag_answer(q)
    print(f"\nQ: {q}")
    print(f"A: {out['answer']}")
    print(f"Fontes: {out['sources']}")

```

Verificação

Ingestão: docs contém ≥ 10 páginas com text não-vazio

Chunking: chunks têm tamanho médio próximo de 800 chars; overlap visível ao comparar chunks adjacentes

Indexing: collection.count() == len(chunks) ; pasta data/chroma/ foi criada

Retrieval: top-5 retornados com distance crescente

Generation: 5 respostas geradas com citação de fonte; pelo menos 3 com citação **correta** (verificável manualmente no PDF)

Troubleshooting

PROBLEMA 1: CHROMADB ERRO DE VERSÃO DE PROTOBUF

Sintoma: ImportError: cannot import name 'builder' from 'google.protobuf.internal'

Solução: uv pip install --force-reinstall chromadb em ambiente limpo. Conflito comum com versões antigas de tensorflow ou langchain.

PROBLEMA 2: EMBEDDINGS RATE-LIMITED

Sintoma: RateLimitError ao indexar grande corpus.

Solução: Reduzir batch automaticamente — Chroma já faz batching mas para corpus >1000 chunks usar `embedding_function` com `retry` custom ou processar em lotes manuais com `time.sleep(1)` entre eles.

PROBLEMA 3: RESPOSTAS SEMPRE “NAO ENCONTRADO NO CORPUS”

Sintoma: Mesmo perguntando sobre tema central do corpus, modelo nega.

Causa: `distance` dos top-5 está muito alta (>0.5) — corpus não cobre o tema **ou** chunks estão fragmentados demais.

Solução: Inspecionar `hits` manualmente. Se distances altas, aumentar `chunk_size` para 1200 ou usar `chunk_overlap=200`. Se chunks parecem corretos mas LLM nega, suavizar o prompt: "Se a informacao for parcial, responda com o que houver e marque [INCERTO]."

Referências

- [Chroma Quick Start](#)
- [OpenAI Embeddings Guide](#)
- [LangChain Text Splitters](#)

Gerado por `/edu-lab-designer` — 2026-05-18

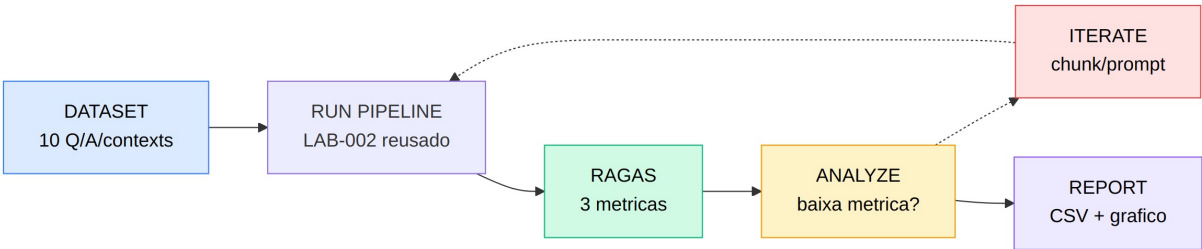
Lab 3: Suite de Avaliação Automatizada com RAGAS

Tipo: Guiado | **Duração:** 60 min | **Nível:** Intermediário | **Bloom:** Evaluate

Objetivos de Aprendizagem:

- **M2-O3:** Avaliar a qualidade de respostas RAG com métricas de retrieval (`precision@k`, `recall@k`) e geração (`faithfulness`, `answer relevancy`)

Fluxo do Lab



Fluxo do Lab 3: dataset de avaliação, execução do pipeline RAG, métricas RAGAS, análise de casos ruins e report

Pré-requisitos

Requisito	Versão
Pipeline do LAB-002 funcionando	—
ragas	0.2+
datasets	2.x
pandas	2.x
langchain-google-genai	2.x (apenas Opção A)

RAGAS usa LLM-as-judge — reuse o **mesmo provider** do LAB-002. Setup completo em `notebooks/API-KEYS.pdf`.

Setup (manter provider do LAB-002)

```
uv pip install ragas datasets pandas matplotlib
```

Reusar o `collection` e função `rag_answer` do LAB-002.

```
import os
import pandas as pd
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision
```

Opção A — Gemini (judge = `gemini-2.5-flash`, qualidade > `flash-lite` para julgamento):

```
uv pip install langchain-google-genai
```

```
from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper

judge_llm = LangchainLLMWrapper(
    ChatGoogleGenerativeAI(model="gemini-2.5-flash", google_api_key=os.environ["GEMINI_API_KEY"])
)
judge_embed = LangchainEmbeddingsWrapper(
    GoogleGenerativeAIEmbeddings(model="models/gemini-embedding-001",
                                   google_api_key=os.environ["GEMINI_API_KEY"])
)
```

Opção B — OpenAI (judge default da RAGAS, não precisa configurar):

```
## RAGAS usa OPENAI_API_KEY do .env automaticamente; judge default = gpt-4o-mini
judge_llm = None
judge_embed = None
```

Atenção quota: 10 queries × 3 métricas faz ~90-150 chamadas internas do judge. Em Gemini free, isso consome ~50% do RPD diário do `gemini-2.5-flash` (250 RPD). Rodar a suite uma única vez por aluno; se precisar iterar, pular para `gemini-2.5-flash-lite` como judge (qualidade menor mas 1000 RPD).

Etapa 1 — Construir Dataset de Avaliação

10 perguntas com **ground-truth** (resposta esperada) montadas manualmente a partir do corpus do LAB-002.

```

EVAL_QUERIES = [
    {
        "question": "Qual a diferenca entre fine-tuning e RAG?",
        "ground_truth": "Fine-tuning atualiza pesos do modelo com dados novos; RAG injeta contexto recuperado  
tempo de inferencia sem alterar pesos.",
    },
    {
        "question": "O que e chunk_overlap em chunking?",
        "ground_truth": "Numero de tokens compartilhados entre chunks adjacentes para preservar continuidade  
semantica em fronteiras.",
    },
    {
        "question": "Qual o papel do top-k em retrieval?",
        "ground_truth": "Numero de chunks mais similares retornados pelo vector store para serem usados como  
contexto.",
    },
    {
        "question": "Por que usar cosine similarity em busca semantica?",
        "ground_truth": "Mede angulo entre vetores normalizados; invariante a magnitude e adequada para embe  
densos.",
    },
    {
        "question": "O que e faithfulness em RAG eval?",
        "ground_truth": "Fracao das afirmacoes da resposta que sao suportadas pelo contexto recuperado.",
    },
    {
        "question": "Como reduzir alucinacao em LLM?",
        "ground_truth": "Restringir resposta ao contexto via prompt, usar temperatura baixa, validar fontes  
citar referencias.",
    },
    {
        "question": "Qual a vantagem de hybrid retrieval?",
        "ground_truth": "Combina BM25 (keyword) e vector search (semantica) capturando match exato e parafrase.",
    },
    {
        "question": "O que e re-ranking com cross-encoder?",
        "ground_truth": "Segunda passada que pontua relevancia (query, doc) juntos; mais caro mas mais preci  
bi-encoder.",
    },
    {
        "question": "Quando usar embeddings menores como all-MiniLM?",
        "ground_truth": "Quando latencia e custo dominam e o ganho de qualidade de embeddings grandes nao  
compensa.",
    },
    {
        "question": "O que mede context precision?",
        "ground_truth": "Proporcao dos chunks recuperados que sao realmente relevantes para a query.",
    },
]

```

Etapa 2 — Rodar Pipeline e Coletar Respostas + Contextos

```

records = []
for item in EVAL_QUERIES:
    out = rag_answer(item["question"], k=5)
    # Recuperar tambem os textos dos contextos (nao so source)
    hits = retrieve(item["question"], k=5)
    # RAGAS 0.2+ usa schema canonico: user_input / response / retrieved_contexts / reference
    records.append({
        "user_input": item["question"],
        "response": out["answer"],
        "retrieved_contexts": [h["text"] for h in hits],
        "reference": item["ground_truth"],
    })

ds = Dataset.from_list(records)
print(ds)

```

Etapa 3 — Calcular Métricas RAGAS

```

## Opção A (Gemini): passar judge_llm e judge_embed configurados no Setup
## Opção B (OpenAI): omitir os 2 últimos argumentos
result = evaluate(
    ds,
    metrics=[faithfulness, answer_relevancy, context_precision],
    llm=judge_llm,          # Opção A: judge_llm; Opção B: None (default)
    embeddings=judge_embed, # Opção A: judge_embed; Opção B: None (default)
)

df = result.to_pandas()
# to_pandas() devolve colunas canonicas: user_input / response / retrieved_contexts / reference
print(df[["user_input", "faithfulness", "answer_relevancy", "context_precision"]])
print("\nMédias:")
print(df[["faithfulness", "answer_relevancy", "context_precision"]].mean())

```

Ponto de verificação: As 3 métricas devem retornar valores em [0, 1]. Médias típicas para corpus bem dimensionado: faithfulness ≥ 0.8 , answer_relevancy ≥ 0.85 , context_precision ≥ 0.6 .

Etapa 4 — Analisar Casos Ruins

Identificar a query com **pior** faithfulness e diagnosticar:

```

worst = df.sort_values("faithfulness").iloc[0]
print(f"Pior pergunta: {worst['user_input']}")
print(f"Faithfulness: {worst['faithfulness']:.3f}")
print(f"Resposta: {worst['response']}")
print(f"\nContextos recuperados:")
for i, c in enumerate(worst["retrieved_contexts"]):
    print(f"    [{i}] {c[:200]}...")

```

Reflexão:

- A resposta tem afirmações **fora** dos contextos? (faithfulness baixa = alucinação parcial)
- Os contextos eram relevantes mas não suficientes?
- Vale ajustar `chunk_size` ou aumentar `k` ?

Etapa 5 — Report CSV + Gráfico

```
import matplotlib.pyplot as plt

df.to_csv("ragas_report.csv", index=False)

fig, ax = plt.subplots(figsize=(10, 4))
df[["faithfulness", "answer_relevancy", "context_precision"]].plot(
    kind="bar", ax=ax, alpha=0.85
)
ax.set_xticklabels(range(1, len(df) + 1))
ax.set_xlabel("Query #")
ax.set_ylabel("Score")
ax.set_ylim(0, 1.05)
ax.set_title("RAGAS - 3 métricas por query")
ax.legend(loc="lower right")
plt.tight_layout()
plt.savefig("ragas_report.png", dpi=150)
plt.show()
```

Verificação

Dataset: 10 entradas com `question`, `answer`, `contexts`, `ground_truth`

3 métricas RAGAS calculadas sem erro

Médias documentadas em texto livre (3 linhas mínimas)

Pior caso analisado com diagnóstico (chunk, prompt, ou k)

CSV + PNG gerados em disco

PEER MINI-CHECKPOINT (MAT-019)

Em duplas, 5 min cada:

1. Mostrar o gráfico de barras.
2. Apresentar a estratégia de chunking escolhida e justificar com 1 número observado (ex.: "context_precision médio = 0.62; aumentei `k` para 7 e subiu para 0.71").
3. Receber feedback estruturado: 1 ponto forte + 1 risco.

Troubleshooting

PROBLEMA 1: RAGAS RECLAMA DE OPENAI KEY

Solução: `os.environ["OPENAI_API_KEY"]` precisa estar populado **antes** de importar `ragas`. RAGAS usa LLM-as-judge internamente.

PROBLEMA 2: MÉTRICAS TODAS IGUAIS A `NAN`

Sintoma: Coluna inteira retorna NaN.

Causa: Dataset mal-formatado — `contexts` precisa ser **lista de strings**, não lista única.

Solução: Verificar `type(records[0]["contexts"])` — deve ser `list`. Cada elemento string.

PROBLEMA 3: FAITHFULNESS MUITO ALTA (>0.95) SUSPEITA

Causa: Possivelmente o judge LLM está sendo permissivo demais. Validar manualmente 2-3 casos: se a resposta tem afirmação não-suportada pelo contexto, RAGAS deveria marcar < 1.0.

Solução: Trocar judge model: `evaluate(ds, metrics=[...], llm=ChatOpenAI(model="gpt-4o"))` para julgamento mais rigoroso.

Referências

- [RAGAS Docs](#)
- [Metrics Overview](#)

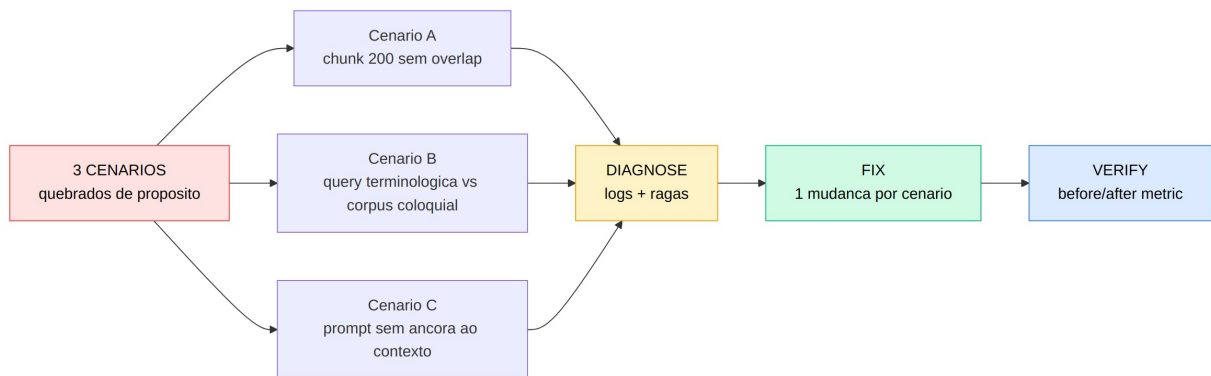
Lab 4: Diagnóstico de Falhas RAG

Tipo: Challenge | Duração: 50 min | Nível: Intermediário | Bloom: Analyze

Objetivos de Aprendizagem:

- M2-O4:** Diagnosticar falhas comuns de RAG (chunk mal dimensionado, retrieval mismatch, alucinação, contexto irrelevante) usando logs e métricas

Fluxo do Lab



Fluxo do Lab 4: diagnóstico de 3 cenários de falha RAG quebrados de propósito

Pré-requisitos

- Pipeline RAG do LAB-002 e suite de eval do LAB-003 funcionando
- 3 corpus alternativos (fornecidos pelo instrutor) que reproduzem cada falha

Reusar **mesma opção** (A/B/C) escolhida no LAB-002 — não trocar provider no meio do M2. Setup completo em `notebooks/API-KEYS.pdf`.

Setup

Reusar `client`, `LLM_MODEL`, `EMBED_MODEL` e `embed_fn` configurados no LAB-002 (qualquer das 3 opções).

```
import os
from pathlib import Path
import chromadb
## Reusar rag_answer, retrieve, ragas evaluate do LAB-002/003
## (embed_fn já está configurado conforme Opcao A/B/C escolhida)

chroma = chromadb.PersistentClient(path="data/chroma_diag")
```

Observação para Opção A (Gemini): o cenário B (retrieval mismatch) já é robusto com `gemini-embedding-001` — pode ser difícil reproduzir queda significativa. Para forçar o cenário, trocar temporariamente para `sentence-transformers/all-MiniLM-L6-v2` (mais sensível a paráfrases).

Cenário A — Chunk Mal Dimensionado (chunks de 200 chars sem overlap)

Indexar o mesmo corpus do LAB-002 mas com `chunk_size=200, overlap=0`.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter_bad = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=0)

## (re-ingest com splitter_bad em collection "papers_chunk200")
## rodar 3 perguntas do dataset de eval do LAB-003 e medir faithfulness
```

Hipótese a confirmar: Faithfulness cai porque conceitos com 1 frase explicativa ficam **partidos** entre chunks; LLM recebe pedaço incompleto e completa por palpite.

Tarefa:

1. Rodar 3 perguntas do dataset de eval.
2. Comparar faithfulness média com a do LAB-003.
3. Inspeccionar **um** chunk recuperado e mostrar a fragmentação.

Cenário B — Retrieval Mismatch (vocabulário query ≠ vocabulário corpus)

Usar corpus técnico (`papers/`) com queries em linguagem coloquial deliberadamente vaga.

Queries:

- “Como o computador aprende sozinho?” → corpus usa “unsupervised learning”
- “Pra que serve aquele negocio de vetor de palavras?” → corpus usa “word embeddings”

Hipótese a confirmar: Context precision cai porque embedding model não casa “computador aprende sozinho” com “unsupervised learning” via similaridade densa apenas.

Tarefa:

1. Rodar as 2 queries no pipeline do LAB-002.
2. Inspeccionar `hits` — top-5 são relevantes?
3. Reescrever as queries usando termos técnicos do corpus e re-medir.

Cenário C — Alucinação por Prompt Sem Âncora

Mesmo corpus, mas usar **prompt sem instrução de ancoragem**:

```
LOOSE_PROMPT = """Responda a pergunta com base nos textos abaixo.

{context}

PERGUNTA: {question}
"""
```

(versus o prompt do LAB-002 que dizia “Responda APENAS com base no contexto” + “Se nao houver, diga Nao encontrado”).

Query: “Quantos parametros tem o GPT-5?” (corpus não tem essa informação).

Hipótese a confirmar: Sem âncora, LLM inventa um número plausível; faithfulness vai a zero na detecção manual.

Tarefa:

1. Rodar a query com `LOOSE_PROMPT`.
2. Registrar a resposta exata.
3. Trocar para o prompt original (com âncora + “Não encontrado”) e re-rodar.

Etapa Final — Tabela de Diagnóstico

Preencher a tabela após rodar os 3 cenários:

Cenário	Falha Observada	Métrica Afetada	Fix Aplicado	Métrica Depois
A	Fragmentação de conceitos	faithfulness ↓	chunk_size=800, overlap=100	faithfulness ↑
B	Top-5 irrelevantes	context_precision ↓	query rewrite + termos técnicos	context_precision ↑
C	Resposta inventada	faithfulness = 0	prompt com âncora + fallback “Não encontrado”	resposta correta de falha

Verificação

- 3 cenários executados com métricas registradas
- Diagnóstico textual para cada cenário (≥3 linhas/cenário) identificando a causa
- 1 fix por cenário aplicado e métrica re-medida
- Tabela final preenchida

Troubleshooting

PROBLEMA 1: CENÁRIO A NÃO MOSTRA QUEDA SIGNIFICATIVA

Causa: O corpus pode ter conceitos suficientemente curtos para sobreviver a chunk=200. Tentar uma query sobre conceito com explicação longa (>400 chars contínuos).

PROBLEMA 2: CENÁRIO B JÁ RECUPERA BEM MESMO COM QUERY COLOQUIAL

Causa: text-embedding-3-small é robusto. Para reforçar o cenário, usar all-MiniLM-L6-v2 (menor) que falha mais em paráfrases. Ou usar BM25 puro para mostrar o caso extremo.

PROBLEMA 3: CENÁRIO C — LLM DIZ “NÃO ENCONTRADO” MESMO COM LOOSE_PROMPT

Causa: Modelos novos (GPT-4o, Claude Sonnet 4.x) têm anchoring forte por default. Trocar para gpt-3.5-turbo ou modelo mais antigo, ou usar pergunta sobre fato muito específico que LLM não tem incentivo a “chutar”. Ex.: “Em que ano o autor publicou a v2.3 desta biblioteca?”.

Referências

- RAG Failure Modes — Pinecone Blog
- Prompting for RAG — Anthropic Cookbook

Gerado por /edu-lab-designer — 2026-05-18

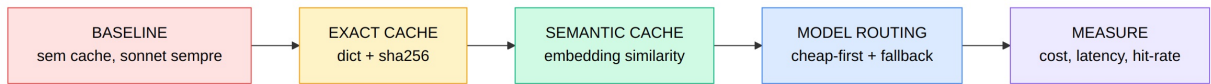
Lab 5: Semantic Cache + Cheap-First Model Routing

Tipo: Guiado | Duração: 75 min | Nível: Intermediário | Bloom: Apply

Objetivos de Aprendizagem:

- M3-O3: Implementar caching (semantic/exact) e model routing (cheap-first com fallback) reduzindo custo de inferência em >50% no projeto

Fluxo do Lab



Fluxo do Lab 5: baseline sem cache, exact-match cache, semantic cache e model routing cheap-first

Pré-requisitos

Requisito	Versão
openai (SDK)	latest
numpy	1.26+
acesso a 2 modelos com price card (cheap + premium)	—

Este lab pede um provider pago real — o ponto pedagógico é medir custo USD por requisição. Setup completo de provider em `notebooks/API-KEYS.pdf` (Plano B: OpenAI/Anthropic; Plano C: local). Se não tiver budget, peça uma chave compartilhada ao instrutor.

Setup (escolha 1 stack)

```
import hashlib
import os
import time
import numpy as np
from openai import OpenAI
from dataclasses import dataclass, field
```

Opção A — OpenAI (\$5 free credit cobre este lab ~3×):

```
client = OpenAI()
EMBED_CLIENT = client
CHEAP_MODEL = "gpt-4o-mini"
PREMIUM_MODEL = "gpt-4o"
EMBED_MODEL = "text-embedding-3-small"

## USD por 1M tokens (verificar https://openai.com/api/pricing antes da turma)
PRICE_INPUT = {CHEAP_MODEL: 0.15, PREMIUM_MODEL: 2.50}
PRICE_OUTPUT = {CHEAP_MODEL: 0.60, PREMIUM_MODEL: 10.00}
```

Opção B — Anthropic (\$5 free credit; clamar no Day-1 — expira em 14 dias):

```
uv pip install anthropic
```



```

## A Anthropic expoe um endpoint OpenAI-compatible: reusamos o mesmo SDK openai no chat.
## Atencao: a Anthropic NAO oferece embeddings – o cache semantico (Etapa 4) precisa de um
## provider de embeddings separado (OpenAI ou sentence-transformers local).
client = OpenAI(
    api_key=os.environ["ANTHROPIC_API_KEY"],
    base_url="https://api.anthropic.com/v1/",
)
CHEAP_MODEL = "claude-haiku-4-5"
PREMIUM_MODEL = "claude-sonnet-4-6"

## Embeddings via OpenAI (Anthropic nao tem endpoint de embeddings)
MBED_CLIENT = OpenAI() # usa OPENAI_API_KEY
MBED_MODEL = "text-embedding-3-small"

## USD por 1M tokens (verificar https://www.anthropic.com/pricing antes da turma)
PRICE_INPUT = {CHEAP_MODEL: 1.00, PREMIUM_MODEL: 3.00}
PRICE_OUTPUT = {CHEAP_MODEL: 5.00, PREMIUM_MODEL: 15.00}

```

Opção C — Plano B sem cartão (Gemini Free como cheap + paid como premium):

```

client = OpenAI(
    api_key=os.environ["GEMINI_API_KEY"],
    base_url="https://generativelanguage.googleapis.com/v1beta/openai/",
)
MBED_CLIENT = client
CHEAP_MODEL = "gemini-2.5-flash-lite"
PREMIUM_MODEL = "gemini-2.5-pro"
MBED_MODEL = "gemini-embedding-001"

## Plano B: substituir "custo USD" por "% de RPD consumido"
## Ainda valida a hipótese arquitetural (cheap-first reduz consumo de quota), mas perde o eixo $.
## Para a tabela final, preencher a coluna "Custo" com "RPD consumido / 250 (flash) ou 100 (pro)"
PRICE_INPUT = {CHEAP_MODEL: 0.0, PREMIUM_MODEL: 0.0} # ambos free, contar por quota
PRICE_OUTPUT = {CHEAP_MODEL: 0.0, PREMIUM_MODEL: 0.0}

```

Recomendação do instrutor: Opção A é a padrão deste lab. Plano C só se o aluno não tiver nenhum cartão internacional disponível — comunicar ao instrutor antes do Day-3.

Etapa 1 — Workload Sintético

Construir 50 queries com **30% de repetição parafraseada** (típico de produção).

```

BASE_QUERIES = [
    "O que é RAG?",
    "Como funciona embedding?",
    "Diferença entre fine-tuning e RAG",
    "Quais são as 3 métricas principais de avaliação RAG?",
    "Como reduzir custo de LLM em produção?",
]

PARAPHRASES = {
    "O que é RAG?": ["Pode me explicar RAG?", "RAG significa o que?", "Como descrever RAG?"],
    "Como funciona embedding?": ["Como embeddings são gerados?", "Embedding é o que?"],
    "Como reduzir custo de LLM em produção?": ["Como economizar em LLM?", "Custos de LLM, como cortar?"],
}

WORKLOAD = []
for base in BASE_QUERIES:
    WORKLOAD.append(base)
    WORKLOAD.extend(PARAPHRASES.get(base, []))
    WORKLOAD.append(base + " (segunda vez exata)")

import random
random.seed(42)
random.shuffle(WORKLOAD)
print(f"Workload: {len(WORKLOAD)} queries")

```

Etapa 2 — Baseline (sem cache, sempre Premium)

```

@dataclass
class CallStats:
    model: str
    input_tokens: int
    output_tokens: int
    latency_ms: float
    cache_hit: str = "miss" # exact | semantic | miss

def call_llm(query: str, model: str = PREMIUM_MODEL) -> tuple[str, CallStats]:
    start = time.perf_counter()
    resp = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": query}],
        temperature=0.0,
    )
    latency = (time.perf_counter() - start) * 1000
    stats = CallStats(
        model=model,
        input_tokens=resp.usage.prompt_tokens,
        output_tokens=resp.usage.completion_tokens,
        latency_ms=latency,
    )
    return resp.choices[0].message.content, stats

def cost_usd(stats: CallStats) -> float:
    return (
        stats.input_tokens * PRICE_INPUT[stats.model] / 1e6
        + stats.output_tokens * PRICE_OUTPUT[stats.model] / 1e6
    )

## Baseline run
baseline_stats = []
for q in WORKLOAD:
    _, st = call_llm(q, model=PREMIUM_MODEL)
    baseline_stats.append(st)

baseline_cost = sum(cost_usd(s) for s in baseline_stats)
baseline_p95 = np.percentile([s.latency_ms for s in baseline_stats], 95)
print(f"BASELINE cost: ${baseline_cost:.4f} | P95 latency: {baseline_p95:.0f}ms")

```

Etapa 3 — Exact-Match Cache

```

class ExactCache:
    def __init__(self):
        self.store: dict[str, str] = {}

    def key(self, query: str) -> str:
        return hashlib.sha256(query.encode()).hexdigest()

    def get(self, query: str) -> str | None:
        return self.store.get(self.key(query))

    def put(self, query: str, answer: str) -> None:
        self.store[self.key(query)] = answer

def run_with_exact_cache():
    cache = ExactCache()
    stats_list = []
    for q in WORKLOAD:
        hit = cache.get(q)
        if hit:
            stats_list.append(CallStats(model="cache", input_tokens=0, output_tokens=0,
                                         latency_ms=0.5, cache_hit="exact"))
        else:
            ans, st = call_llm(q, model=PREMIUM_MODEL)
            cache.put(q, ans)
            stats_list.append(st)
    return stats_list

exact_stats = run_with_exact_cache()
exact_cost = sum(cost_usd(s) for s in exact_stats if s.model != "cache")
exact_hits = sum(1 for s in exact_stats if s.cache_hit == "exact")
print(f"EXACT cache: ${exact_cost:.4f} ({1 - exact_cost/baseline_cost:.0%} de reducao) | hits: {exact_hits}/{len(WORKLOAD)}")

```

Ponto de verificação: Exact cache pega ~10-15% das queries (apenas as “segunda vez exata”).

Etapa 4 — Semantic Cache (similaridade de embedding)

```

class SemanticCache:
    def __init__(self, threshold: float = 0.93):
        self.threshold = threshold
        self.queries: list[str] = []
        self.embeddings: list[np.ndarray] = []
        self.answers: list[str] = []

    def _embed(self, text: str) -> np.ndarray:
        r = EMBED_CLIENT.embeddings.create(model=EMBED_MODEL, input=text)
        return np.array(r.data[0].embedding)

    def get(self, query: str) -> str | None:
        if not self.queries:
            return None
        e = self._embed(query)
        sims = np.array([np.dot(e, em) / (np.linalg.norm(e) * np.linalg.norm(em))
                        for em in self.embeddings])
        idx = int(np.argmax(sims))
        if sims[idx] >= self.threshold:
            return self.answers[idx]
        return None

    def put(self, query: str, answer: str) -> None:
        self.queries.append(query)
        self.embeddings.append(self._embed(query))
        self.answers.append(answer)

def run_with_semantic_cache():
    cache = SemanticCache(threshold=0.93)
    stats_list = []
    for q in WORKLOAD:
        hit = cache.get(q)
        if hit:
            stats_list.append(CallStats(model="cache", input_tokens=0, output_tokens=0,
                                       latency_ms=80, cache_hit="semantic"))
        else:
            ans, st = call_llm(q, model=PREMIUM_MODEL)
            cache.put(q, ans)
            stats_list.append(st)
    return stats_list

sem_stats = run_with_semantic_cache()
sem_cost = sum(cost_usd(s) for s in sem_stats if s.model != "cache")
sem_hits = sum(1 for s in sem_stats if s.cache_hit == "semantic")
print(f"SEMANTIC cache: ${sem_cost:.4f} ({1 - sem_cost/baseline_cost:.0%} de reducao) | hits: {sem_hits}/{len(WORKLOAD)}")

```

Ponto de verificação: Semantic cache deve capturar 25-35% das queries (incluindo paráfrases).

Etapa 5 — Model Routing Cheap-First

```
def classify_complexity(query: str) -> str:
    """Heurística simples – em prod, usar classificador treinado ou LLM rápido."""
    if len(query) < 60 and "?" in query:
        return "simple"
    if any(w in query.lower() for w in ["explique", "compare", "analise", "projete"]):
        return "complex"
    return "simple"

def run_with_routing():
    cache = SemanticCache(threshold=0.93)
    stats_list = []
    for q in WORKLOAD:
        hit = cache.get(q)
        if hit:
            stats_list.append(CallStats(model="cache", input_tokens=0, output_tokens=0,
                                         latency_ms=80, cache_hit="semantic"))
            continue

        complexity = classify_complexity(q)
        model = CHEAP_MODEL if complexity == "simple" else PREMIUM_MODEL
        ans, st = call_llm(q, model=model)
        cache.put(q, ans)
        stats_list.append(st)
    return stats_list

routed_stats = run_with_routing()
routed_cost = sum(cost_usd(s) for s in routed_stats if s.model != "cache")
print(f"ROUTING + semantic: ${routed_cost:.4f} ({1 - routed_cost/baseline_cost:.0%} de reducao)")
```

Meta: Redução $\geq 50\%$ no custo total versus baseline.

Verificação

- Baseline cost** registrado (workload de 50 queries em premium)
- Exact cache** implementado e hit-rate medido
- Semantic cache** implementado com threshold ajustável e hit-rate medido
- Model routing** cheap-first com classificador heurístico
- Tabela final** comparativa abaixo preenchida com números reais

TABELA ESPERADA

Estratégia	Custo Total	Redução vs Baseline	Hit Rate Cache	P95 Latency
Baseline (Premium)	\$X.XXXX	0%	0%	~1800ms
Exact cache	~85% do baseline	~15%	~15%	~1500ms
Semantic cache	~65% do baseline	~35%	~30%	~1300ms
Semantic + Routing	$\leq 50\%$ do baseline	$\geq 50\%$ ✓	~30%	~1100ms

Troubleshooting

PROBLEMA 1: SEMANTIC CACHE HIT-RATE MUITO BAIXO (<10%)

Causa: Threshold alto demais (`0.97+`) — paráfrases não passam.

Solução: Baixar para `0.92-0.93`. Validar manualmente 5 hits para garantir que ainda são semanticamente equivalentes.

PROBLEMA 2: CHEAP MODEL DÁ RESPOSTAS PÉSSIMAS

Sintoma: Respostas curtas e erradas em queries simples.

Solução: Adicionar fallback: se cheap retornar resposta ≤ 30 chars ou contiver “não sei”, chamar premium e cachear o premium.

PROBLEMA 3: CUSTO DO EMBEDDING COMPENSA O GANHO

Causa: Embedding também custa (~\$0.02/1M tokens). Se workload tem poucas queries, semantic cache pode não compensar.

Solução: Combinar exact (free) + semantic (cheap) — sempre tentar exact primeiro, só embeber se exact miss.

Referências

- [OpenAI Pricing](#)
- [Semantic Cache Pattern — Redis](#)
- [Anthropic Prompt Caching](#)

Gerado por `/edu-lab-designer` — 2026-05-18

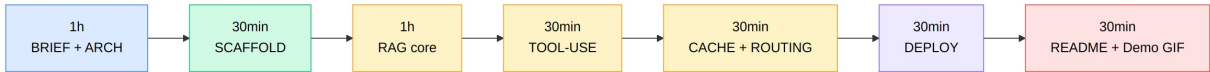
Lab 6: Projeto Portfólio Integrador

Tipo: Projeto | **Duração:** 240 min (4h corridas, 11:00–16:00 do Day-3) | **Nível:** Avançado | **Bloom:** Create

Objetivos de Aprendizagem:

- **M3-O1:** Projetar a arquitetura de uma aplicação LLM-powered considerando custo, latência e observabilidade
- **M3-O2:** Desenvolver um projeto de portfólio individual que integre LLM + RAG + Tool-use sobre um problema escolhido pelo aluno
- **M3-O3:** Implementar caching (semantic/exact) e model routing reduzindo custo de inferência em >50%
- **M3-O4:** Compilar README profissional + demo deployada apto a entrar em portfólio público

Fluxo do Lab



Fluxo do Lab 6: brief e arquitetura, scaffold e etapas do projeto portfólio integrador

Setup de API

Reusar a **opção escolhida** em M1/M2 (Gemini, OpenAI, Anthropic ou Ollama local). Setup completo em `notebooks/API-KEYS.pdf`.

- **Default da turma:** Gemini Free Tier — capacidade suficiente para o capstone se o aluno respeitar 15 RPM.
- **Recomendado para entrega de portfólio público:** Opção A do LAB-005 (OpenAI gpt-4o-mini) — README com custo real em USD pesa mais na banda “Custo/Latência — excelente” da rubrica.
- **Plano B sem cartão:** Gemini Free para retrieval + Gemini Pro paid (na cota free) para premium routing. Documentar no README que o eixo de custo está em “% de quota” em vez de USD.

Problem Brief (Aluno Escolhe na 1ª Hora)

O aluno escolhe **um problema real** que sirva como portfólio público. Critérios:

- Tem um corpus textual disponível (docs internas, livros open-source, papers, regulamentação, etc.)
- Tem ≥ 3 perguntas plausíveis que se beneficiam de LLM + RAG + tool-use
- É demonstrável em ≤ 2 min em um vídeo

Exemplos válidos:

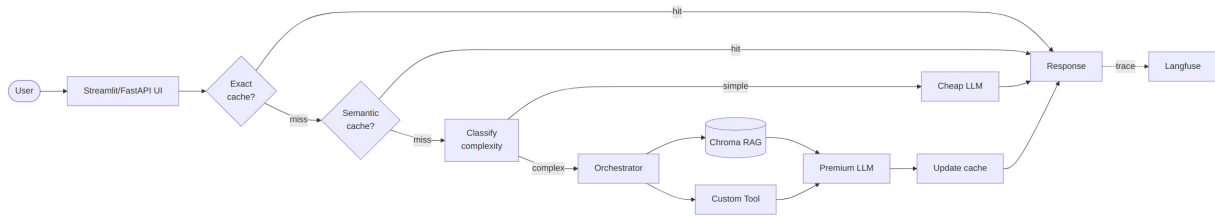
- Assistente de leitura de um livro técnico (corpus = livro, tool = highlight/quote)
- Q&A sobre changelog de uma lib popular (corpus = release notes, tool = check version compat)
- Resumo de podcasts (corpus = transcripts, tool = timestamp lookup)
- Q&A sobre código-fonte (corpus = AST + comments, tool = run snippet)

Exemplos REJEITADOS:

- Chatbot genérico sem corpus específico
- Sumarizador puro (sem retrieval)
- Wrapper de API com 1 endpoint chamando LLM

Etapa 1 — Architecture Design (1h)

Diagramar em 1 página:



Arquitetura do projeto integrador: User, UI, LLM, RAG, tool-use, cache e routing

Entregáveis da etapa:

- 1 diagrama de arquitetura (Mermaid ou Excalidraw)
- Decisão documentada: cheap model, premium model, embedding model, vector store
- Token budget estimado: tokens médios por query × custo por 1M tokens × tráfego esperado

Etapa 2 — Scaffold (30min)

```
projeto-portfolio/  
├── README.md           # placeholder  
├── pyproject.toml       # uv venv  
├── src/  
│   ├── ui/             # streamlit_app.py ou fastapi_app.py  
│   ├── pipeline/  
│   │   ├── rag.py       # ingest, retrieve, augment  
│   │   ├── tools.py      # function-calling registry  
│   │   ├── cache.py      # exact + semantic cache  
│   │   ├── routing.py    # classify_complexity + dispatch  
│   └── observability/  
│       └── trace.py      # langfuse wrapper  
├── data/  
│   ├── corpus/          # documentos fonte  
│   └── chroma/           # vector store persistente (gitignored)  
├── tests/  
│   └── test_eval.py      # mini suite RAGAS  
└── .env.example
```

```
uv init projeto-portfolio  
cd projeto-portfolio  
uv add openai chromadb pypdf langchain-text-splitters streamlit pydantic langfuse pandas ragas
```

Etapa 3 — RAG Core (1h)

Implementar `src/pipeline/rag.py` reusando LAB-002 (ingestion, chunking, embedding, retrieval, generation).

Validar com 3 perguntas do problema escolhido:


```
from src.pipeline.rag import build_rag_pipeline

rag = build_rag_pipeline(corpus_dir="data/corpus")
for q in ["pergunta 1", "pergunta 2", "pergunta 3"]:
    print(rag.answer(q))
```

Etapa 4 — Tool-use (30min)

Definir **1 tool específica** do domínio em `src/pipeline/tools.py` reusando LAB-001.

Exemplos por domínio:

- Livro técnico → `lookup_chapter(chapter: int) -> str`
- Changelog → `check_compat(lib: str, version: str) -> dict`
- Podcasts → `get_timestamp(quote: str) -> str`
- Código → `run_snippet(code: str) -> str` (sandboxed)

Integrar no orchestrator: para queries complexas que pedem ação além de retrieve, despachar tool.

Etapa 5 — Cache + Routing (30min)

Implementar `src/pipeline/cache.py` e `src/pipeline/routing.py` reusando LAB-005.

Meta-bar: rodar workload sintético de 30 queries e mostrar redução $\geq 50\%$ vs baseline-no-cache:

```
python -m src.bench compare
```

Output esperado: tabela CSV com colunas `strategy`, `cost_usd`, `p95_ms`, `cache_hit_rate`.

Etapa 6 — Deploy (30min)

Escolher **uma** opção:

Opção	Quando usar	Tempo de setup
Streamlit Cloud	Demo público rápido, UI mínima	~10 min
HuggingFace Spaces	Demo permanente com custom domain	~15 min
FastAPI + Docker + Fly.io	Produção real, API REST	~25 min

Mínimo aceitável: URL pública acessível, sem crash em fluxo principal, segredos via env var.

Etapa 7 — README + Demo GIF (30min)

`README.md` deve ter (na ordem):

1. **Title + 1 sentence pitch**
2. **Demo GIF** (10-15s, terminal ou UI)
3. **Live URL**
4. **Problem statement** (3 linhas)
5. **Architecture** (diagrama da Etapa 1)
6. **Setup** (`uv sync && cp .env.example .env && streamlit run src/ui/streamlit_app.py`)
7. **Cost & Latency** (tabela da Etapa 5)
8. **Design Decisions** (3-5 bullets: por que esse embedding, esse chunking, esse router)
9. **Limitations** (3 bullets honestos)

Demo GIF: usar `peek`, `terminalizer`, ou OBS Studio + `ffmpeg` para gerar GIF leve (<5MB).

Verificação (Rubrica 3-Bandas — MAT-028)

Critério	Peso	Básico (60-70)	Sólido (75-85)	Excelente (90-100)
Técnica	40%	LLM + RAG + tool-use funcionam isoladamente	+ integração end-to-end com erros tratados + logs estruturados	+ arquitetura justificada + código testado + eval automatizada do retrieval
README	30%	problem statement + setup	+ diagrama de arquitetura + métricas observadas	+ decisões de design + GIF/demo embedado + custo por requisição
Custo/Latência	20%	mede custo por chamada	+ cache implementado com hit-rate medido	+ routing cheap-first com redução $\geq 50\%$ comprovada + P95 reportado
Demo	10%	endpoint/URL acessível	+ 1 fluxo demonstrável sem crash	+ UX cuidada (streaming, loading states) + link público com TTL declarado

Apresentação (16:00–17:00, MAT-029): 5 min por aluno + 2 min Q&A peer.

Troubleshooting

PROBLEMA 1: CUSTO DO BASELINE FOI PEQUENO DEMAIS; $\geq 50\%$ DE REDUÇÃO NÃO É DEMONSTRÁVEL

Causa: Workload pequeno; flutuação dominante.

Solução: Aumentar workload para 50+ queries com $\geq 30\%$ de paráfrases. Ou aceitar a meta “redução qualitativamente visível” e documentar no README a limitação.

PROBLEMA 2: DEPLOY QUEBROU NO ÚLTIMO MINUTO

Causa: Secrets não migraram, dependência nativa (faiss) não instalou em ambiente cloud.

Solução: Plano B documentado — gravar vídeo do demo local (<2 min) e linkar no README. **Não** mascarar a falha; documentar no `Limitations`.

PROBLEMA 3: RAGAS EVAL AUTOMATIZADA NÃO RODA NO CI

Causa: Falta `OPENAI_API_KEY` no CI; ou suite custa muito por execução.

Solução: Marcar suite como `pytest -m eval` e rodar manualmente; commitar resultado em `evals/results-YYYY-MM-DD.json` versionado. Isso ainda conta como “eval automatizada”.

Referências

- [Streamlit Deployment](#)
- [HuggingFace Spaces](#)
- [Langfuse Docs](#)
- [README Best Practices — GitHub Docs](#)